



Universidad Autónoma de Nuevo León
Facultad de Ingeniería Mecánica y
Eléctrica

Laboratorio Temas Selectos de Sistemas
Inteligentes
PIA

Docente: Raquel Martínez Martínez

Nombre	Matricula	Brigada	Salón	Hora
Patricio Alessandro Milán Gámez	1970626	515	4205	V-V4
Francisco Yahir Martinez Servin	1956693	(No lleva lab)	-	-

Ene-Jun 2025

01/Mayo/2025

ÍNDICE

Introducción.....	3
Resumen	3
Metodología utilizada	3
Desarrollo	4
Dataset.....	4
Desarrollo del Modelo de Red Neuronal	5
Programa de detección de mascarillas	8
Diseño del circuito.....	10
Funcionamiento	11
Conexión física	11
Resultados	12
Componentes del Sistema	14
Hardware	14
Software.....	14
Ejecución del Programa en Python	14
Video de funcionamiento	15
Conclusión.....	15
Bibliografía	15

Introducción

El presente proyecto tiene como objetivo desarrollar un sistema inteligente capaz de detectar si una persona porta o no una mascarilla, utilizando visión por computadora y una red neuronal artificial. Este sistema fue diseñado para funcionar en tiempo real, reconociendo rostros a través de una cámara y clasificando su estado mediante un modelo previamente entrenado.

Una red neuronal artificial es un modelo computacional inspirado en el funcionamiento del cerebro humano, compuesto por capas de nodos interconectados que permiten identificar patrones complejos en los datos. Este tipo de modelos es una de las bases del machine learning, una rama de la inteligencia artificial que permite a las máquinas aprender a partir de datos, sin ser programadas explícitamente para cada tarea. En conjunto, forman un sistema inteligente, es decir, un sistema capaz de percibir su entorno, procesar información, tomar decisiones y actuar de manera autónoma con base en su análisis.

A lo largo del proyecto se integraron técnicas de procesamiento de imágenes, entrenamiento de redes neuronales y comunicación con hardware (Arduino) para activar respuestas físicas como LEDs, sonido o mecanismos de acceso, en función de los resultados de la detección.

Resumen

Este proyecto consiste en un sistema que detecta si una persona está usando mascarilla mediante una cámara y una red neuronal entrenada. El sistema analiza en tiempo real el rostro captado por la cámara y determina si la persona lleva o no mascarilla. Si detecta una mascarilla, envía una señal al Arduino, que puede activar funciones como encender un LED, emitir un sonido o permitir el paso mediante un mecanismo. Es una combinación de inteligencia artificial y electrónica que automatiza decisiones basadas en el reconocimiento visual.

Metodología utilizada

1. Recolección y preparación del dataset

Se utilizó un conjunto de imágenes etiquetadas con dos clases: con mascarilla y sin mascarilla. Las imágenes fueron redimensionadas y convertidas a escala de grises para facilitar el entrenamiento del modelo.

2. Entrenamiento del modelo con red neuronal

Se empleó un clasificador basado en reconocimiento facial con la técnica LBPH (Local Binary Patterns Histograms). El modelo fue entrenado con las imágenes previamente procesadas y guardado en un archivo .xml para su uso posterior.

3. Implementación del sistema de visión por computadora

Con ayuda de OpenCV y MediaPipe, se desarrolló un programa que capta video en tiempo real, detecta los rostros presentes y los evalúa utilizando el modelo entrenado. Según la predicción, se determina si la persona lleva mascarilla o no.

4. Comunicación con Arduino

El sistema envía una señal al microcontrolador Arduino a través del puerto serial. Dependiendo del resultado del reconocimiento, se transmite una letra específica ('P' para con mascarilla, 'N' para sin mascarilla).

5. Control de dispositivos electrónicos

El Arduino recibe la señal y activa los componentes del circuito, como LEDs, zumbadores o mecanismos de apertura. Esto permite una respuesta física inmediata basada en el análisis visual del sistema inteligente.

Desarrollo

Dataset

El Dataset fue obtenido de un usuario de Kaggle que previamente trabajo las imágenes aumentando su tamaño para que cada clase tuviera una distribución equitativa, eliminando las imágenes con ruido que podrían considerarse valores atípicos y descartando imágenes que podrían desequilibrar el modelo.

En un principio las imágenes fueron obtenidas de la fuente anteriormente mencionada, luego se volvió a aplicar detección facial con MediaPipe face



Ilustración 1. Ejemplos de imágenes de dataset con mascarilla y sin mascarilla

detection. Una vez ubicado el rostro en la imagen se procedió a recortar esta área y se la redimensionó.

En la carpeta “dataset” existen dos directorios: “with_mask” y “without_mask”. Cada uno de estos posee 2994 imágenes a color de rostros con mascarilla y sin mascarilla respectivamente. Cada una de ellas posee 70 pixeles de alto y ancho.

Desarrollo del Modelo de Red Neuronal

Primero se inicializó el entorno virtual de Python donde se instalarán las librerías correspondientes para trabajar con el proyecto.

Librerías necesarias:

- OpenCV para el procesamiento de imágenes.
- numpy para manejar arreglos numéricos.
- Mediapipe para trabajar con las imágenes.

```
PS C:\Users\patmi\OneDrive\Documentos\ProyectoRedesNeuronales\DeteccionMascarillaIntegrandoArduino> env\Scripts\activate
(env) PS C:\Users\patmi\OneDrive\Documentos\ProyectoRedesNeuronales\DeteccionMascarillaIntegrandoArduino> pip freeze
absl-py==2.2.1
attrs==25.3.0
cffi==1.17.1
contourpy==1.3.1
cycler==0.12.1
flatbuffers==25.2.10
fonttools==4.56.0
jax==0.5.3
jaxlib==0.5.3
kiwisolver==1.4.8
matplotlib==3.10.1
mediapipe==0.10.21
ml_dtypes==0.5.1
numpy==1.26.4
opencv-contrib-python==4.11.0.86
opt_einsum==3.4.0
packaging==24.2
pillow==11.1.0
protobuf==4.25.6
pycparser==2.22
pyparsing==3.2.3
python-dateutil==2.9.0.post0
scipy==1.15.2
sentencepiece==0.2.0
six==1.17.0
sounddevice==0.5.1
```

Ilustración 2. Listado de librerías instaladas específicamente para este proyecto.

Se desarrolló el código para entrenar un modelo de reconocimiento facial utilizando el algoritmo Local Binary Pattern Histogram (LBPH), con el objetivo de detectar rostros con o sin mascarilla.

1. Se especifica la ruta donde se encuentran las imágenes utilizadas para entrenar el modelo. Estas imágenes están organizadas en subdirectorios, cada uno representando una clase.
2. Se crean dos listas:
 - labels: para almacenar las etiquetas de cada imagen.
 - facesData: para almacenar los datos de las imágenes en sí.
3. Para cada imagen dentro del subdirectorio:
 - Se forma su ruta completa.
 - Se lee en escala de grises usando `cv2.imread(image_path, 0)`.
 - Se agrega la imagen a la lista facesData.
 - Se almacena la etiqueta correspondiente en labels.
 - Se incrementa label al cambiar de subdirectorio.
4. Se instancia el reconocedor facial `"cv2.LBPHFaceRecognizer_create()"` y se entrena con los datos recopilados (facesData) y sus etiquetas (labels).

5. El modelo entrenado se guarda en un archivo llamado "face_mask_model.xml", lo que permitirá su reutilización sin necesidad de entrenarlo nuevamente en futuras ejecuciones.

```
1 import cv2
2 import os
3 import numpy as np
4
5 #Ruta del directorio que almacena las imagenes
6 dataPath = "C:/Users/patmi/OneDrive/Documentos/ProyectoRedesNeuronales/DeteccionMascarillaIntegrandoArduino/Dataset"
7
8 #Lista de subdirectorios en dataPath
9 dir_list = os.listdir(dataPath)
10 print("Lista de archivos", dir_list) #Muestra los subdirectorios
11
12 labels = [] #Lista para almacenar las etiquetas
13 facesData = [] #Lista para almacenar las imagenes
14 label = 0
15
16 for name_dir in dir_list:
17     #Formar la ruta completa del subdirectorio actual
18     dir_path = dataPath + "/" + name_dir
19
20     for file_name in os.listdir(dir_path):
21         #Formar la ruta completa de cada imagen en el subdirectorio
22         image_path = dir_path + "/" + file_name
23         #print(image_path) #Muestra la ruta completa de la imagen
24         #Leer imagen en escala de grises
25         image = cv2.imread(image_path, 0)
26
27         #cv2.imshow("Image", image) #Muestra las imagenes
28         #cv2.waitKey(10)
29
30         #Agregar todas las imagenes en facesData y las etiquetas en labels
31         facesData.append(image) labels.append(label)
32
33     label += 1 #Se asigna una nueva etiqueta a cada subdirectorio
34
35 print("Etiqueta 0: ", np.count_nonzero(np.array(labels) == 0))
36 print("Etiqueta 1: ", np.count_nonzero(np.array(labels) == 1))
37
38 #Local Binary Path History Face Recognizer
39 face_mask = cv2.face.LBPHFaceRecognizer_create()
40
41 #ENTRENAMIENTO
42 print("ENTRENANDO...")
43 face_mask.train(facesData, np.array(labels))
44
45 #Almacenar modelo
46 face_mask.write("face_mask_model.xml")
47 print("Modelo almacenado")
```

Ilustración 4. Código del modelo de entrenamiento.

```
PS C:\Users\patmi\OneDrive\Documentos\ProyectoRedesNeuronales\DeteccionMascarillaIntegrandoArduino> & c:/Users/patmi/OneDrive/Documentos/ProyectoRedesNeuronales/DeteccionMascarillaIntegrandoArduino/env/Scripts/python.exe c:/Users/patmi/OneDrive/Documentos/ProyectoRedesNeuronales/DeteccionMascarillaIntegrandoArduino/train.py
Lista de archivos ['without_mask', 'with_mask']
Etiqueta 0: 2994
Etiqueta 1: 2994
ENTRENANDO...
Modelo almacenado
```

Ilustración 3. Salida en terminal del programa.

Programa de detección de mascarillas

- Se establece una lista con las etiquetas "without_mask" y "with_mask", y crea un objeto para reconocer patrones faciales usando el método LBPH. También define la ruta del archivo donde está guardado el modelo.
- Se define una función para verificar si el archivo del modelo existe y, si existe, intenta cargarlo. Si no lo encuentra, muestra un error.
- Se abre la cámara web de la computadora y utiliza la librería mediapipe para identificar dónde hay rostros en cada imagen que captura la cámara.
- Para cada rostro que se encuentra:
 - Recortar la parte de la imagen donde está el rostro.
 - Convertir esa imagen a blanco y negro y la hace más pequeña.
 - Utilizar el modelo cargado previamente para predecir si ese rostro tiene o no una mascarilla.
 - Dibujar un recuadro alrededor del rostro y escribe una etiqueta (con o sin mascarilla) con un color diferente según la predicción.
- Mostrar en la pantalla lo que está viendo la cámara con los recuadros y las etiquetas dibujadas.
- El programa se mantiene funcionando hasta que el usuario presiona la tecla "ESC".


```

1 import cv2
2 import mediapipe as mp
3 import os
4
5
6
7
8
9 def find_model(model_path):
10     if os.path.exists(model_path):
11         try:
12             face_mask.read(model_path)
13             print("Modelo cargado exitosamente.")
14         except Exception as e:
15             print(f"Error al cargar el modelo: {e}")
16     else:
17         print("Error: No se encontró el archivo 'face_mask_model.xml")
18
19 def detect_face_mask():
20     cap = cv2.VideoCapture(0, cv2.CAP_DSHOW)
21     with mp_face_detection.FaceDetection(min_detection_confidence=0.5) as face_detection:
22         while True:
23             ret, frame = cap.read()
24             if not ret:
25                 print("ERROR: No se pudo acceder a la cámara")
26                 break
27
28             frame = cv2.flip(frame, 1) #Voltea la visualización en espejo
29
30             height, width, _ = frame.shape
31             frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
32             results = face_detection.process(frame_rgb)
33
34             if results.detections is not None:
35                 for detection in results.detections:
36                     xmin = int(detection.location_data.relative_bounding_box.xmin * width)
37                     ymin = int(detection.location_data.relative_bounding_box.ymin * height)
38                     w = int(detection.location_data.relative_bounding_box.width * width)
39                     h = int(detection.location_data.relative_bounding_box.height * height)
40
41                     if xmin < 0 or ymin < 0:
42                         continue
43
44                     try:
45                         face_image = frame[ymin : ymin + h, xmin : xmin + w]
46                         face_image = cv2.cvtColor(face_image, cv2.COLOR_BGR2GRAY)
47                         face_image = cv2.resize(face_image, (70, 70), interpolation=cv2.INTER_CUBIC)
48                         result = face_mask.predict(face_image)
49
50                         cv2.putText(frame, "{}".format(result), (xmin, ymin - 5), 1, 1.3, (210, 124, 176), 1,
51 cv2.LINE_AA)
52
53                         if result[1] < 150:
54                             color = (0, 255, 0) if LABELS[result[0]] == "with_mask" else (0, 0, 255)
55                             cv2.putText(frame, "{}".format(LABELS[result[0]]), (xmin, ymin - 25), 2, 1, color, 1,
56 cv2.LINE_AA)
57                             cv2.rectangle(frame, (xmin, ymin), (xmin + w, ymin + h), color, 2)
58                         except:
59                             print("Calibrando...")
60
61                         #Mensaje para cierre de ventana
62                         cv2.putText(frame, "Presiona 'ESC' para finalizar", (10, 30), cv2.FONT_HERSHEY_SIMPLEX, 1, (255, 255, 255),
63 2, cv2.LINE_AA)
64
65                         cv2.imshow("Frame", frame)
66                         if cv2.waitKey(1) == 27: # Presionar 'ESC' para salir
67                             break
68
69                     cap.release()
70                     cv2.destroyAllWindows()
71
72 if __name__ == "__main__":
73     mp_face_detection = mp.solutions.face_detection
74     LABELS = ["without_mask", "with_mask"]
75     face_mask = cv2.face.LBPHFaceRecognizer_create()
76     model_path = "face_mask_model.xml"
77
78     find_model(model_path)
79     detect_face_mask()

```

Ilustración 5. Código del programa de detección de mascarilla

Diseño del circuito

LEDs

- LED Verde (conectado al pin digital 7): Se enciende cuando se detecta que una persona lleva mascarilla.
- LED Rojo (conectado al pin digital 8): Se enciende cuando se detecta que una persona no lleva mascarilla.

Buzzer Pasivo

- Está conectado al pin digital 9.
- Puede sonar diferente según si se detecta un error (sin mascarilla) o una acción correcta (con mascarilla).

Servomotor

- Conectado al pin digital 6 del Arduino.
- El servo puede abrir o cerrar una compuerta o barrera:
 - Con mascarilla: se abre la compuerta (gira el servo).
 - Sin mascarilla: la compuerta permanece cerrada.

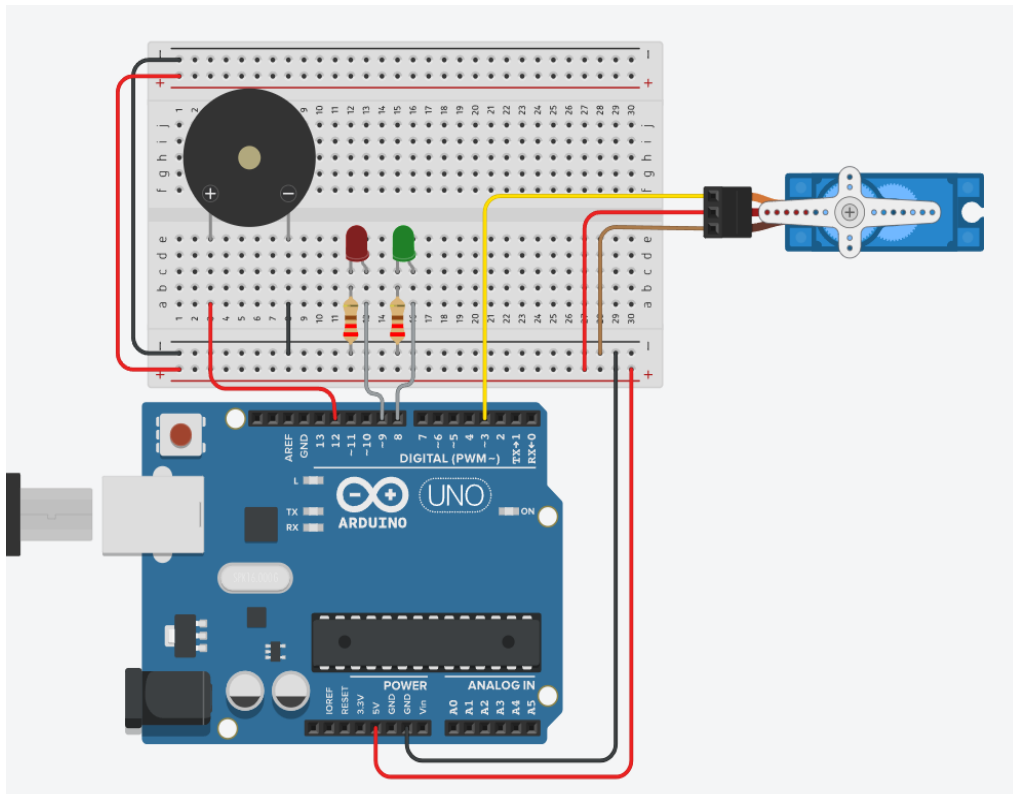


Ilustración 6. Diseño en simulación del sistema.

Funcionamiento

Este circuito forma parte de un sistema de detección de mascarilla controlado por un programa en Python que envía datos por puerto serial (USB). Según la señal enviada:

'P' (Persona con mascarilla):

- Se enciende el LED verde
- Se emite un sonido de confirmación en el buzzer
- El servo se mueve (por ejemplo, para abrir una puerta)

'N' (Persona sin mascarilla):

- Se enciende el LED rojo
- Se emite un sonido de alerta o advertencia
- El servo no se mueve o vuelve a la posición cerrada

Conexión física

Se siguió el diseño de la simulación después de testarlo para realizar las conexiones correctas.

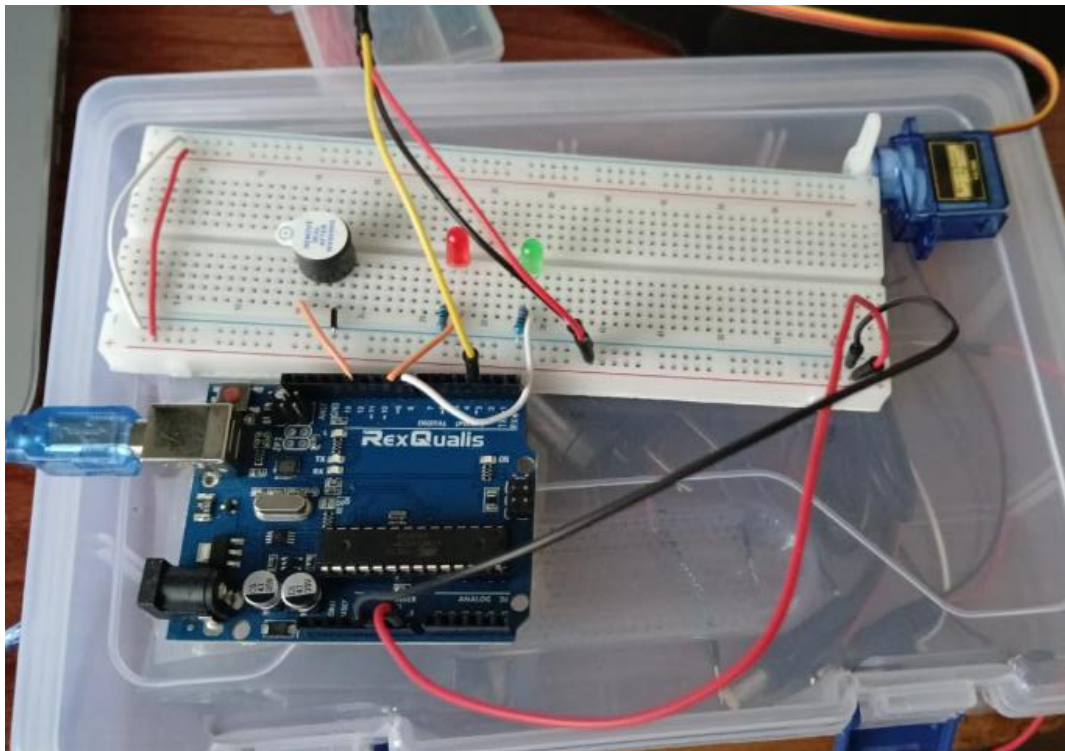


Ilustración 7. Conexión del circuito en físico.

Resultados

Si detecta un rostro sin mascarilla o mascarilla mal puesta:

Si detecta un rostro con mascarilla:

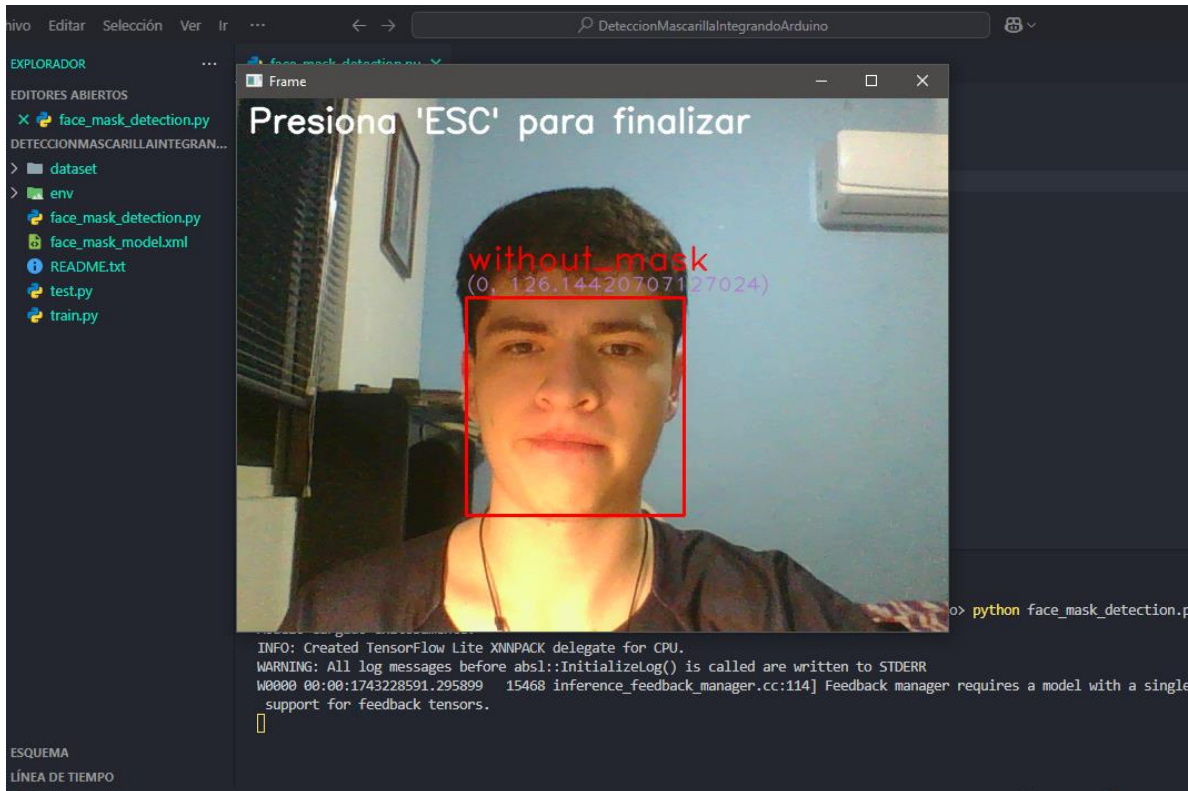


Ilustración 9. Resultado de imagen sin mascarilla.

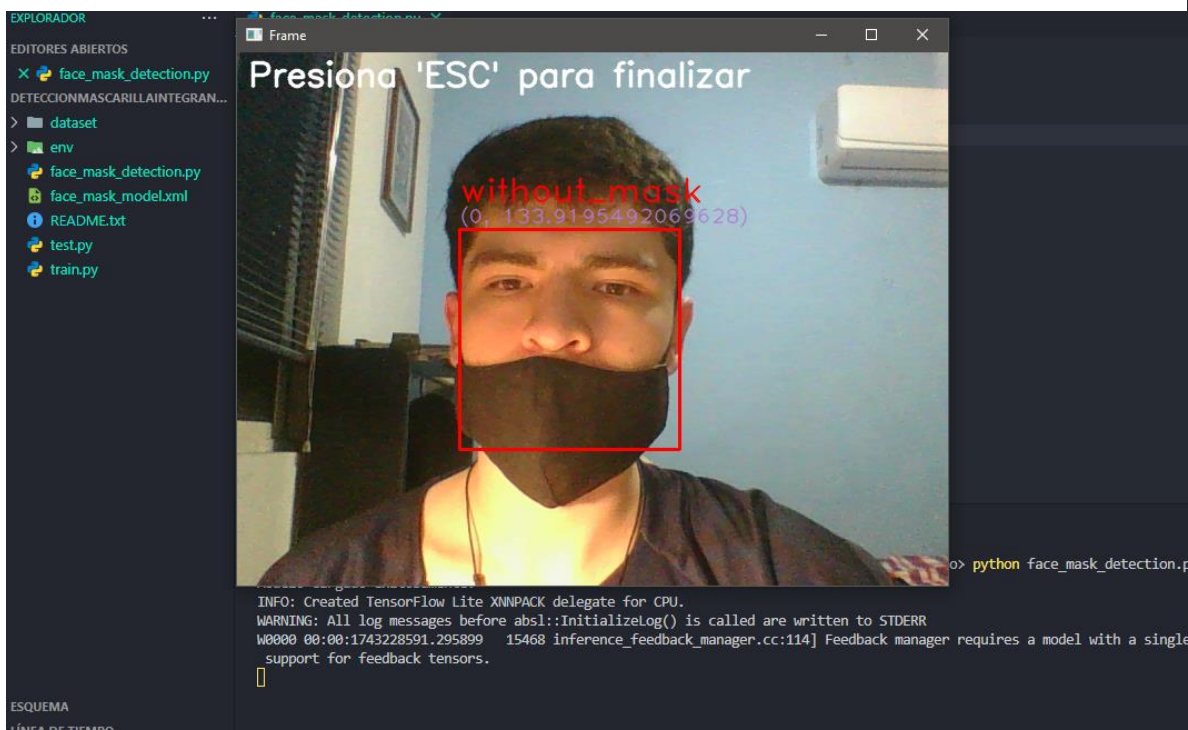


Ilustración 9. Resultado de imagen con mascarilla mal puesta.

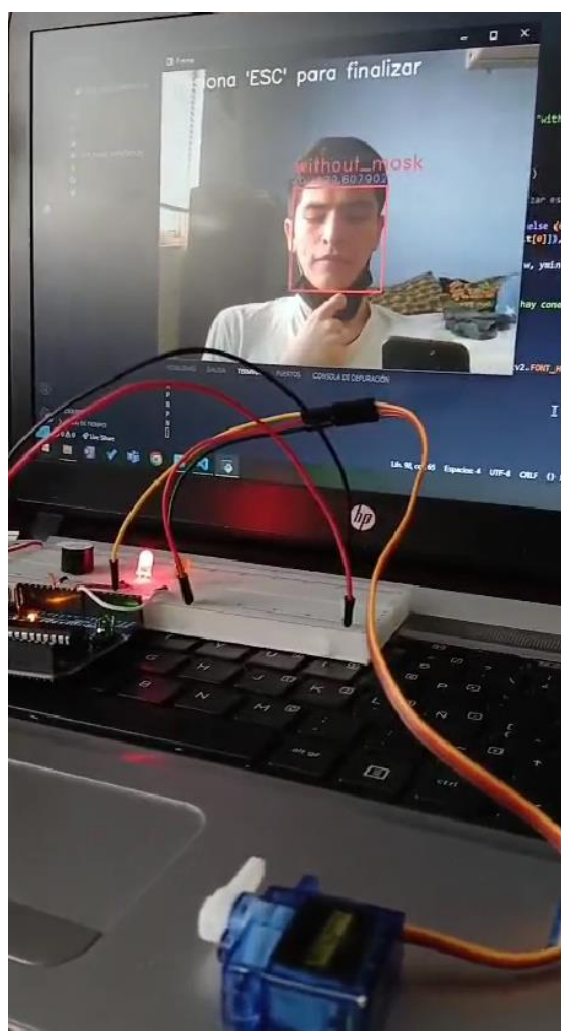
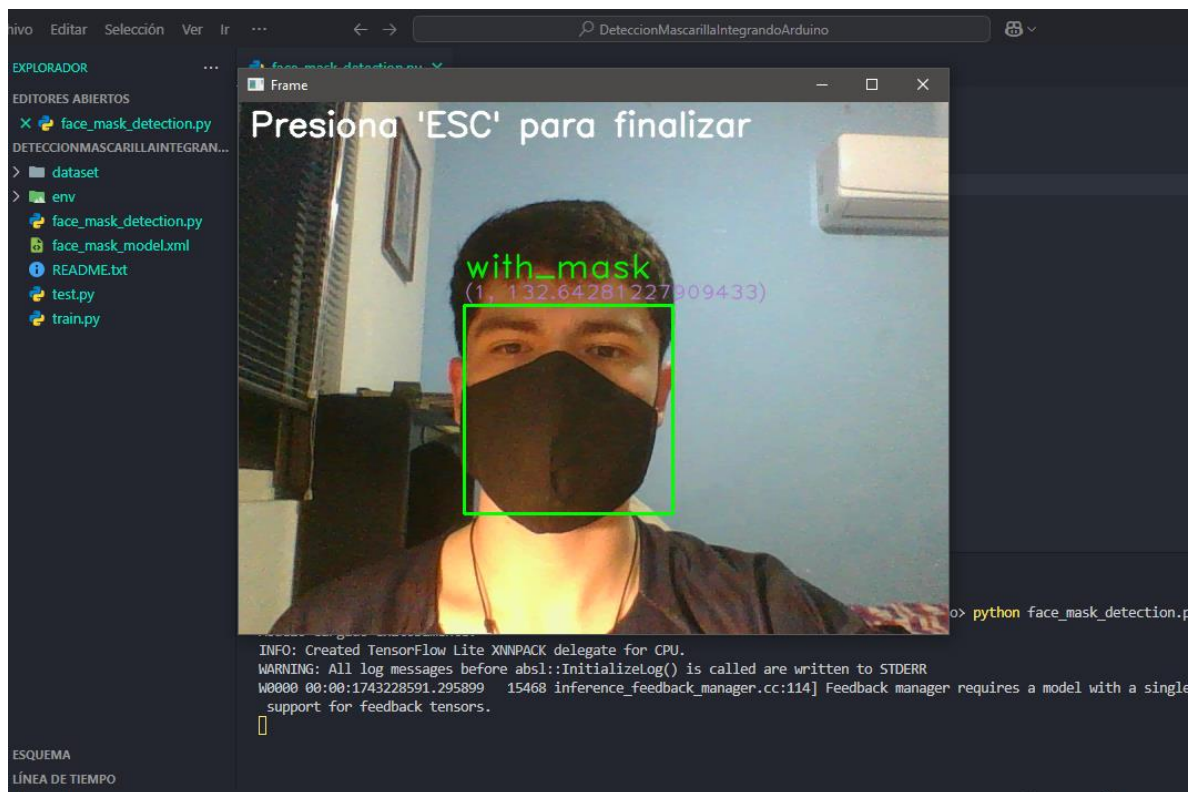


Ilustración 10. Funcionamiento del sistema en estado Sin mascarilla

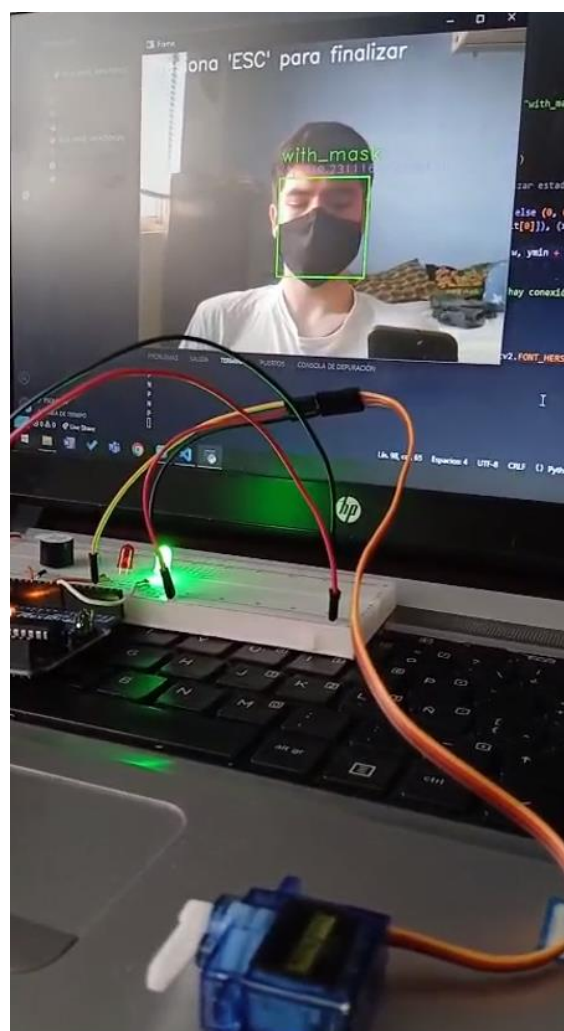


Ilustración 11. Funcionamiento del sistema en estado Con mascarilla

Componentes del Sistema

Hardware

- Arduino UNO / Nano / Mega
- Cables Dupont (M/M)
- Protoboard
- LEDs (verde y rojo)
- Zumbador pasivo
- Resistencias de 220Ω
- Fuente de alimentación (USB o batería)
- Computadora con puerto USB
- Cámara Web

Software

- Python 3
- OpenCV
- MediaPipe
- PySerial
- NumPy
- Modelo entrenado (face_mask_model.xml)

Ejecución del Programa en Python

python detect_mask_system.py

Video de funcionamiento

<https://youtube.com/shorts/4h1bF6EE3KM?feature=share>

Conclusión

En conclusión, nuestro proyecto de detección de mascarillas mediante una red neuronal e integración con Arduino es una combinación de visión por computadora y control de hardware permite implementar sistemas autónomos capaces de emitir alertas o tomar decisiones sin intervención humana directa. El sistema puede aplicarse en diversos entornos donde el control sanitario es crítico, como hospitales, oficinas gubernamentales, centros educativos, fábricas, transporte público y eventos masivos. En cuanto a los alcances, el sistema actualmente permite la detección básica de mascarillas y la activación de alertas visuales y sonoras, pero este proyecto puede escalarse e integrarse con mecanismos automatizados de acceso, sistemas de registro de datos, almacenamiento en la nube o incluso con bases de datos de empleados para registrar cumplimiento sanitario.

Bibliografía

Face mask detection. (2021, 19 mayo). Kaggle.

<https://www.kaggle.com/datasets/vijaykumar1799/face-mask-detection>

Prado, K. S. D. (2018, 19 junio). Face Recognition: Understanding LBPH Algorithm - TDS Archive - Medium. Medium. <https://medium.com/data-science/face-recognition-how-lbph-works-90ec258c3d6b>

SPSS Modeler Subscription. (2024). <https://www.ibm.com/docs/es/spss-modeler/saas?topic=networks-neural-model>

Prado, K. S. D. (2018, 19 junio). Face Recognition: Understanding LBPH Algorithm - TDS Archive - Medium. Medium. <https://medium.com/data-science/face-recognition-how-lbph-works-90ec258c3d6b>